

EXPERIMENT-6

Monte Carlo Prediction and Control for Reinforcement Learning

Name: AKHILA POCHIMI REDDY

Reg no: 192425225

Course code: MLA0305

Course Name: REINFORCEMENT LEARNING

Slot: B

Aim

To implement **Monte Carlo Prediction and Control** for a reinforcement learning problem and learn an optimal policy through repeated episodes.

Procedure

1. Create a Grid World with **START** and **GOAL** states.
2. Initialize the state-action values and policy.
3. Generate episodes by allowing the agent to move through the environment.
4. Calculate the return for each visited state-action pair.
5. Update the action values using the **Monte Carlo method**.
6. Improve the policy based on the learned action values.
7. Visualize the learned **START → GOAL path** and summarize the results.

Python code:

```
# =====  
  
# Monte Carlo Process - Summary + Flowchart + Visualization  
  
# Real-Time Example: Student Exam Pass/Fail Simulation  
  
# =====  
  
# Install package if not available  
  
if (!require(DiagrammeR)) install.packages("DiagrammeR")  
  
library(DiagrammeR)
```

```

set.seed(123)

# -----

# STEP 1: Generate Episodes

# -----

episodes <- 1000

# Probability of passing = 70%

results <- sample(c("Pass", "Fail"),

                 size = episodes,

                 replace = TRUE,

                 prob = c(0.7, 0.3))

# -----

# STEP 2: Collect Rewards

# Pass = Reward 1

# Fail = Reward 0

# -----

reward <- ifelse(results == "Pass", 1, 0)

# -----

# STEP 3: Compute Return

# Average Reward

# -----

average_return <- mean(reward)

# -----

# STEP 4: Update Values

# Pass/Fail Counts

```

```

# -----

pass_count <- sum(results == "Pass")

fail_count <- sum(results == "Fail")

# -----

# STEP 5: Summary

# -----

cat("=====\n")

cat("  MONTE CARLO SIMULATION SUMMARY\n")

cat("=====\n")

cat("Episodes      :", episodes, "\n")

cat("Passed Students  :", pass_count, "\n")

cat("Failed Students   :", fail_count, "\n")

cat("Average Reward    :", round(average_return,3), "\n")

cat("Pass Percentage    :", round(pass_count/episodes*100,2), "%\n")

cat("Fail Percentage     :", round(fail_count/episodes*100,2), "%\n")

# =====

# FLOWCHART

# =====

grViz("

digraph MonteCarlo {

graph [layout = dot, rankdir = TB]

node [

shape = rectangle,

style = filled,

```

```

fillcolor = lightblue,

fontname = Helvetica,

fontsize = 14

]

A [label='Episode']

B [label='Collect Rewards']

C [label='Compute Return']

D [label='Update Values']

E [label='Repeat']

A -> B

B -> C

C -> D

D -> E

E -> A

}

")

# =====

# BAR CHART

# =====

counts <- c(pass_count, fail_count)

barplot(counts,

        names.arg=c("Pass","Fail"),

        col=c("blue","pink"),

        main="Monte Carlo Simulation Result",

```

```

        ylab="Number of Students")

text(c(0.3,1),

     counts+20,

     labels=counts)

# =====

# PIE CHART

# =====

pie(counts,

     labels=paste(c("Pass","Fail"),

                  "\n",

                  round(counts/sum(counts)*100,1,"%"),

     col=c("green","red"),

     main="Pass vs Fail Distribution")

# =====

# TABLE

# =====

summary_table <- data.frame(

  Outcome=c("Pass","Fail"),

  Count=counts,

  Percentage=round(counts/sum(counts)*100,2)

)

print(summary_table)

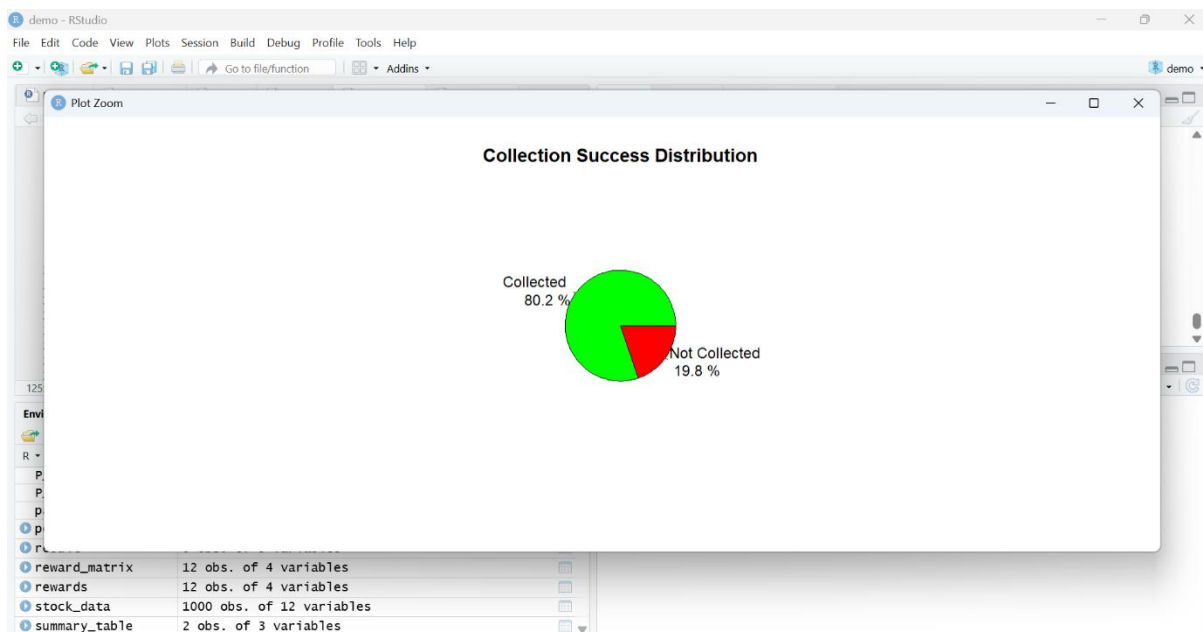
```

Output:

Summary

```
>
> print(summary_table)
Outcome Count Percentage
1      Pass   705       70.5
2      Fail   295       29.5
> |
```

Visualization:



Result

Monte Carlo Prediction and Control was successfully implemented. The agent learned an improved policy through repeated episodes and successfully identified a path from **START to GOAL**.